COMP 110

Analyzing Data from a CSV in a Jupyter Notebook (Continued)

Announcements

EX09: Data Analysis for Continuous Improvement

- Due on LDOC!
- Can work with a partner
- Will use the functions/tools we discussed last class + today

Final Exam 4 pm April 30

Make-up Final will be at 12pm on Friday, May 1 in SN014

- If you qualify for the makeup, fill out the makeup request form ASAP
- You won’t get a confirmation until next week

EX09

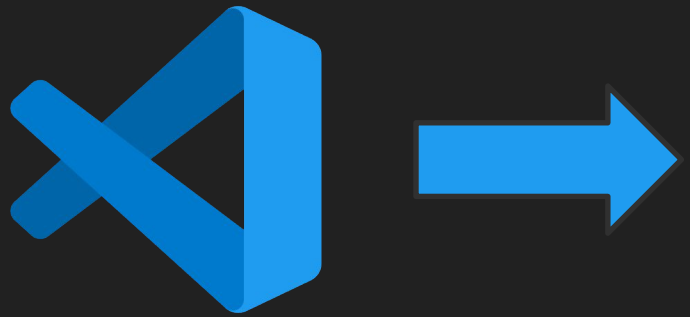
Today's goal: Make sure you've made good progress on starting EX09

Pull from Github

Make sure you push your current work first!!!!

Follow the instructions on the writeup under "Getting Started"

Getting Familiar with The Files



Markdown in Jupyter Notebooks (Cheat Sheet here!)

Syntax	Renders as
`# Heading 1`	**Large section title**
`## Heading 2`	**Subsection title**
`**bold text**`	**bold text**
`*italic text*`	*italic text*
`- item\n- item`	• Bulleted list
`1. first\n2. sec`	1. Numbered list
`` `inline code` ``	`monospace snippet`

What is a CSV?

A .csv file stores tabular data as plain text. Each line is a row, and values in that row are separated by commas.

column

column headers

row of data

```
year,temp,result
2018,48.24,no shadow
2019,46.63,shadow
2020,52.50,shadow
2021,45.86,no shadow
2022,50.36,shadow
```

read_csv_rows

Before calling read_csv_rows (data in our .csv):

```
year,temp,result
2018,48.24,no shadow
2019,46.63,shadow
2020,52.50,shadow
2021,45.86,no shadow
2022,50.36,shadow
```

After calling read_csv_rows (Python objects in our .ipynb):

```
[{'year': '2018', 'temp': '48.24', 'result': 'no shadow'}
{'year': '2019', 'temp': '46.63', 'result': 'shadow'},
{'year': '2020', 'temp': '52.50', 'result': 'shadow'},
{'year': '2021', 'temp': '45.86', 'result': 'no shadow'},
{'year': '2022', 'temp': '50.36', 'result': 'shadow'}]
```

`column_values` function

Example input:

```
[{'year': '2018', 'temp': '48.24', 'result': 'no shadow'}
{'year': '2019', 'temp': '46.63', 'result': 'shadow'},
{'year': '2020', 'temp': '52.50', 'result': 'shadow'},
{'year': '2021', 'temp': '45.86', 'result': 'no shadow'},
{'year': '2022', 'temp': '50.36', 'result': 'shadow'}]
```

+

"temp"

Returns:

```
['48.24','46.63','52.50','45.86','50.36']
```

`columnar` function definition

<u>Purpose:</u> *Transform* a table represented as a list of rows (e.g. `list[dict[str, str]]`) into one represented as a dictionary of columns (e.g. `dict[str, list[str]]`).

Example input:

```
[{'year': '2018', 'temp': '48.24', 'result': 'no shadow'}
{'year': '2019', 'temp': '46.63', 'result': 'shadow'},
{'year': '2020', 'temp': '52.50', 'result': 'shadow'},
{'year': '2021', 'temp': '45.86', 'result': 'no shadow'},
{'year': '2022', 'temp': '50.36', 'result': 'shadow'}]
```

Returns:

```
{'year': ['2018', '2019', '2020', '2021', '2022'],
'temp': ['48.24', '46.63', '52.50', '45.86', '50.36'],
'result': ['no shadow', 'shadow', 'shadow', 'no shadow', 'shadow']}
```

Diagramming Different Types

```
data_rows: list[dict[str, str]] = [{'name': 'zinnia', 'plant': 'apr'}, {'name': 'joepye', 'plant': 'jan'}]
data_cols: dict[str, list[str]] = {'name': ['zinnia', 'joepye'], 'plant': ['apr', 'jan']}
```

Diagramming Different Types

```
data_rows: list[dict[str, str]] = [{'name': 'zinnia', 'plant': 'apr'}, {'name': 'joepye', 'plant': 'jan'}]
data_cols: dict[str, list[str]] = {'name': ['zinnia', 'joepye'], 'plant': ['apr', 'jan']}
```

**** You should be able to diagram and write code to handle nested data structures like these for the final!***

`head` function definition

<u>Purpose:</u> Produce a new column-based table with only the first **`N`** (a parameter) rows of data for each column.

<u>Why:</u> Visualizing a table with hundreds, thousands, or millions of rows in it is overwhelming. You frequently want to just see the first few rows of a table to get a sense you are on the correct path.

- Function name: **`head`**
- Parameters:

  1. **`dict[str, list[str]]`** - a column-based table of data that *will not be mutated* (so think of the **`data_cols`** structure we just diagrammed!)

  2. **`int`** - The number of "rows" to include in the resulting list (this is your **`N`**)

- Return type: **`dict[str, list[str]]`**